Sesión 6

Árboles de Decisión y Random Forest

Carlos César Sánchez Coronel

MACHINE LEARNING

Contenido

1. Introducción
2. Árboles de Decisión
3. Random Forest
4. Comparación de Métricas
5. Casos de Uso
6. Implementación en scikit-learn
7. Caso Avanzado: Detección de Fraude
8. Conclusión

Contenido

1. Introducción
2. Árboles de Decisión
3. Random Forest
4. Comparación de Métricas
5. Casos de Uso
6. Implementación en scikit-learn
7. Caso Avanzado: Detección de Fraude
8. Conclusión

Modelos basados en reglas

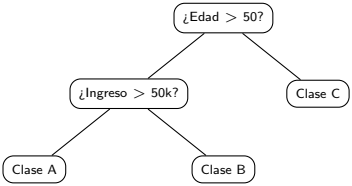
- Los árboles de decisión son modelos predictivos que representan reglas en forma jerárquica.
- A diferencia de los modelos lineales (que suponen una relación paramétrica global), los árboles **particionan el espacio** en regiones más simples.
- **Ventaja principal**: interpretabilidad.
- **Problema**: un solo árbol tiene alta varianza (sobreajuste) y es inestable.
- **Solución**: combinar múltiples árboles con **Random Forest** (ensamble basado en bagging).

Contenido

1. Introducción
2. Árboles de Decisión
3. Random Forest
4. Comparación de Métricas
5. Casos de Uso
6. Implementación en scikit-learn
7. Caso Avanzado: Detección de Fraude
8. Conclusión

Estructura de un árbol

- **Nodo raíz**: contiene todos los datos.
- **Nodos internos**: evalúan una característica y bifurcan.
- **Ramas**: resultados de la evaluación (sí/no, umbral).
- **Nodos hoja**: contienen la predicción final (clase o valor).
- **Profundidad**: número de niveles desde la raíz a una hoja.



Criterios de división: Clasificación

Índice de Gini

$$G = 1 - \sum_{k=1}^{K} p_k^2$$

- Mide la probabilidad de clasificar erróneamente un elemento si se etiqueta al azar según la distribución del nodo.
- $G = 0$: nodo puro.

Entropía

$$H = -\sum_{k=1}^{K} p_k \log_2 p_k$$

- Mide el desorden o incertidumbre.
- $H = 0$: nodo puro; máxima en distribución uniforme.

Ejemplo (60 % A, 40 % B)

Gini $= 1 - (0{,}6^2 + 0{,}4^2) = 0{,}48$
Entropía $= -(0{,}6 \log_2 0{,}6 + 0{,}4 \log_2 0{,}4) \approx 0{,}971$

Ganancia de información

Definición

$$IG(D, a) = H(D) - \sum_{v \in Valores(a)} \frac{|D_v|}{|D|} H(D_v)$$

Mide la reducción de entropía al dividir según el atributo a.

Ejemplo (Pregunta 3)

Nodo padre con $H = 0{,}9$.
División A: hijos con $H = 0{,}3$ (50) y $0{,}4$ (50) $\Rightarrow IG = 0{,}9 - (0{,}5 \cdot 0{,}3 + 0{,}5 \cdot 0{,}4) = 0{,}55$.
División B: hijos con $H = 0{,}1$ (80) y $0{,}6$ (20) $\Rightarrow IG = 0{,}9 - (0{,}8 \cdot 0{,}1 + 0{,}2 \cdot 0{,}6) = 0{,}70$.
Se elige la división B.

Árboles de regresión

- Variable objetivo continua.
- Criterio de división: minimizar la suma de cuadrados residual (RSS):

$$RSS = \sum_{i \in R_1} (y_i - \bar{y}_{R_1})^2 + \sum_{i \in R_2} (y_i - \bar{y}_{R_2})^2$$

- Predicción en cada hoja: $\bar{y}$ de las observaciones en esa hoja.
- Comparación con regresión lineal: más flexible, pero no extrapola.

Sobreajuste y poda

Problema

Árboles sin restricciones crecen hasta hojas puras $\rightarrow$ sobreajuste (100 % en entrenamiento, pobre en prueba).

Poda por complejidad-costo

$$RSS_{\text{podado}} = RSS + \alpha|T|$$

donde $|T|$ = número de hojas, α penaliza la complejidad.

Hiperparámetros de parada

- `max_depth`
- `min_samples_split`
- `min_samples_leaf`

Ventajas y desventajas de los árboles

Ventajas	**Desventajas**
Interpretables y visualizables	Alta varianza (inestables)
No requieren escalado	Propensos al sobreajuste
Manejan mixtura de variables	Pueden ser sesgados con clases desbalanceadas
Robustos a outliers	En regresión, menos precisos que otros modelos

Contenido

1. Introducción
2. Árboles de Decisión
3. Random Forest
4. Comparación de Métricas
5. Casos de Uso
6. Implementación en scikit-learn
7. Caso Avanzado: Detección de Fraude
8. Conclusión

Intuición: combinar múltiples árboles

- Un solo árbol tiene alta varianza.
- Si promediamos muchos árboles **no correlacionados**, la varianza se reduce drásticamente sin aumentar el sesgo.
- Random Forest construye una colección de árboles con dos fuentes de aleatoriedad:
  1. Muestras bootstrap (bagging).
  2. Selección aleatoria de características en cada división.

Bootstrap aggregating (Bagging)

Procedimiento

Para $b = 1, \ldots, B$:

1. Generar una muestra bootstrap (con reemplazo) del mismo tamaño que el original.
2. Entrenar un árbol profundo (sin podar) en esa muestra.

Predicción final:

- Clasificación: voto mayoritario.
- Regresión: promedio.

- Aprox. 63.2 % de las observaciones originales aparecen en cada muestra.
- El 36.8 % restante se llama **out-of-bag (OOB)**.

Aleatoriedad adicional: selección de características

- En cada división de cada árbol, solo se considera un subconjunto aleatorio de características (`max_features`).
- Esto reduce la correlación entre árboles, ya que evita que todos elijan las mismas variables fuertes.
- Valores típicos:
  - Clasificación: $\sqrt{p}$ (raíz cuadrada del total de características).
  - Regresión: $p/3$ (un tercio).

Error Out-of-Bag (OOB)

Definición

Para cada observación i, se predice usando solo los árboles donde i no fue incluida en la muestra bootstrap. El error OOB es el error promedio en esas predicciones.

- Proporciona una estimación del error de prueba **sin necesidad de validación cruzada**.
- Comparable a validación cruzada leave-one-out cuando B es grande.
- Precaución: puede ser optimista si hay dependencias (ej. series temporales).

Hiperparámetros principales

Hiperparámetro	**Qué controla**
`n_estimators`	Número de árboles (a mayor, más estable)
`max_features`	Características por división
`max_depth`	Profundidad máxima de cada árbol
`min_samples_split`	Mínimo de muestras para dividir un nodo
`min_samples_leaf`	Mínimo de muestras en una hoja
`class_weight`	Peso de clases (para desbalance)

Importancia de características

Por impureza (MDI)

- Suma la reducción de impureza (Gini o entropía) aportada por cada variable en todas las divisiones, promediada sobre todos los árboles.
- Sesgo hacia variables numéricas o con muchos valores.

Por permutación (MDA)

- Se permuta aleatoriamente la variable en las muestras OOB y se mide el aumento del error.
- Más fiable, especialmente con variables correlacionadas.

Importancia negativa

Si al permutar una variable el error disminuye, significa que esa variable no es importante y posiblemente añade ruido.

Ventajas y desventajas de Random Forest

Ventajas	**Desventajas**
Menor sobreajuste que un solo árbol	Mayor costo computacional
Muy preciso en clasificación/-regresión	Pérdida de interpretabilidad global
Maneja alta dimensionalidad	Lento en predicción con muchos árboles
Estima error OOB internamente	No extrapola
Importancia de variables	
Robusto a outliers y faltantes (con imputación)	

Contenido

1. Introducción
2. Árboles de Decisión
3. Random Forest
4. Comparación de Métricas
5. Casos de Uso
6. Implementación en scikit-learn
7. Caso Avanzado: Detección de Fraude
8. Conclusión

Clasificación: precisión, recall, F1, AUC

- En problemas desbalanceados (ej. fraude 1 %), la **exactitud (accuracy)** es engañosa.
- Métricas clave:
  - **Recall**: $\frac{VP}{VP+FN}$ (minimizar falsos negativos).
  - **Precisión**: $\frac{VP}{VP+FP}$ (minimizar falsos positivos).
  - **F1-score**: media armónica de ambas.
  - **AUC-ROC**: capacidad discriminativa global.

Regresión: RMSE vs MAE

MAE $= \frac{1}{n}\sum|y_i - \hat{y}_i|$

- Robusto a outliers.
- Todos los errores pesan igual.

RMSE $= \sqrt{\frac{1}{n}\sum(y_i - \hat{y}_i)^2}$

- Penaliza más los errores grandes.
- Sensible a outliers.

Ejemplo (Pregunta 13)

Errores A: [5,5,5,100] $\rightarrow$ MAE=28.75, RMSE50.19
Errores B: [20,20,20,20] $\rightarrow$ MAE=20, RMSE=20
Si los errores grandes son costosos, elegir B.

Contenido

1. Introducción
2. Árboles de Decisión
3. Random Forest
4. Comparación de Métricas
5. Casos de Uso
6. Implementación en scikit-learn
7. Caso Avanzado: Detección de Fraude
8. Conclusión

Detección de fraudes (clases desbalanceadas)

- **Datos**: transacciones, fraude 0.1 %.
- **Random Forest** con `class_weight='balanced'` o SMOTE.
- **Métricas prioritarias**: recall y precisión.
- **Ajuste**: umbral de decisión para equilibrar costos.

Análisis de riesgo crediticio

- **Objetivo**: predecir impago.
- **Random Forest** proporciona importancia de variables (historial crediticio, ingresos).
- Se pueden visualizar árboles individuales para explicar decisiones a auditores.

Predicción de precios de viviendas (regresión)

- **Datos**: área, habitaciones, antigüedad, ubicación, etc.
- **Árbol de regresión**: interpretable pero menos preciso.
- **Random Forest Regressor**: mejora precisión, importancia de características.
- **Limitación**: no extrapola a precios muy altos (fuera del rango de entrenamiento).

Contenido

1. Introducción
2. Árboles de Decisión
3. Random Forest
4. Comparación de Métricas
5. Casos de Uso
6. Implementación en scikit-learn
7. Caso Avanzado: Detección de Fraude
8. Conclusión

Árbol de decisión

```
from sklearn.tree import DecisionTreeClassifier, plot_tree
import matplotlib.pyplot as plt

clf = DecisionTreeClassifier(max_depth=5, min_samples_split=10,
                             criterion='gini')
clf.fit(X_train, y_train)
y_pred = clf.predict(X_test)

# Visualizaci n
plt.figure(figsize=(20,10))
plot_tree(clf, filled=True, feature_names=feature_names,
          class_names=class_names)
plt.show()

# Importancia de caracter sticas
importances = clf.feature_importances_
```

Random Forest

```
from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(n_estimators=100, max_features='sqrt',
                            oob_score=True, class_weight='balanced',
                            random_state=42)
rf.fit(X_train, y_train)
print(f"OOB Score: {rf.oob_score_}")

# Importancia por impureza
importances = rf.feature_importances_

# Importancia por permutaci n (m s fiable)
from sklearn.inspection import permutation_importance
perm_importance = permutation_importance(rf, X_test, y_test,
                                         n_repeats=10)
```

Contenido

1. Introducción
2. Árboles de Decisión
3. Random Forest
4. Comparación de Métricas
5. Casos de Uso
6. Implementación en scikit-learn
7. Caso Avanzado: Detección de Fraude
8. Conclusión

Diseño de un sistema antifraude con Random Forest

- Millones de transacciones/día, fraude 0.1 %.
- Requisitos: rapidez en inferencia, interpretabilidad (para analistas).

Feature engineering

1. Log del monto.
2. Hora del día (codificación cíclica seno/coseno).
3. Agregadas por usuario: frecuencia últimas 24h, monto promedio, desviación.
4. Distancia geográfica y tiempo desde última transacción.
5. Puntuación de anomalía (Isolation Forest) como predictor.

Manejo del desbalance y evaluación

- **Técnicas**: pesos de clase (`class_weight='balanced'`), ajuste de umbral, SMOTE.
- **Métricas**: recall (prioritario), precisión, F1, curva PR.
- **Validación**: usar `StratifiedKFold` para mantener proporción de clases.

Interpretabilidad

- Importancia por permutación para explicar variables globales.
- SHAP para explicar predicciones individuales a analistas.

Monitoreo en producción

- **Métricas de negocio**: recall $\leftrightarrow$ fraude no detectado (pérdidas), precisión $\leftrightarrow$ transacciones legítimas bloqueadas (insatisfacción).
- **Error OOB** mensual para estimar rendimiento sin validación separada.
- **Detectar drift** en distribuciones de características (PSI, KS).

Contenido

1. Introducción
2. Árboles de Decisión
3. Random Forest
4. Comparación de Métricas
5. Casos de Uso
6. Implementación en scikit-learn
7. Caso Avanzado: Detección de Fraude
8. Conclusión

Lo que hemos aprendido

Conceptos clave

- **Árboles de decisión**: estructura, criterios Gini/entropía, regresión, sobreajuste, poda.
- **Random Forest**: bagging, aleatoriedad en características, error OOB, importancia de variables.
- **Comparación de métricas**: RMSE/MAE en regresión, precisión/recall/F1 en clasificación desbalanceada.
- **Casos de uso**: fraude, riesgo crediticio, predicción de precios.

¡Gracias!